

DATA STRUCTURE LAB

TOPIC : SINGLE LINKED LIST

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

Nithiyasri S
192572082RB

Linear Search

LINEAR SEARCH
Create a C program to perform Linear Search on an array of 10 integers. The program should allow manual input or random generation of array elements. It must display the first position of the element (1-based indexing), total comparisons made, search status.

```
7 int choice, target, i;
8 int comparisons = 0;
9 int position = -1;
10 clock_t start, end;
11 double time_taken;
12
13 printf("Enter 1 for manual input, 2 for random: ");
14 scanf("%d", &choice);
15
16 if (choice == 1) {
17     printf("Enter 10 integers:\n");
18     for (i = 0; i < 10; i++) {
19         scanf("%d", &arr[i]);
20     }
21 } else {
22     srand(time(NULL));
23     printf("Randomly generated array:\n");
24     for (i = 0; i < 10; i++) {
25         arr[i] = rand() % 100;
26         printf("%d ", arr[i]);
27     }
```

42
23
67
12
89
34
56
78
90
11
34

OUTPUT
Enter 1 for manual input, 2 for random: Randomly generated array:
33 75 96 10 96 26 78 84 31 7
Enter target value: Target 23 not found in the array
Comparisons made: 10
Status: Not Found

Binary Search

BINARY SEARCH
"Write a C program implementing binary search on a sorted array of 15 integers. The program must: (1) accept an unsorted array and sort it first, (2) perform binary search, (3) display search iterations with mid-point calculations, (4) show final result with comparison count"

```
4 int i, j, temp;
5 for (i = 0; i < n - 1; i++) {
6     for (j = 0; j < n - i - 1; j++) {
7         if (arr[j] > arr[j + 1]) {
8             temp = arr[j];
9             arr[j] = arr[j + 1];
10            arr[j + 1] = temp;
11        }
12    }
13 }
14 }
15
16 int isSorted(int arr[], int n) {
17     int i;
18     for (i = 0; i < n - 1; i++) {
19         if (arr[i] > arr[i + 1])
20             return 0;
21     }
22     return 1;
23 }
```

50 30 10 90 40 70 20
80 60 5 45 35 25 55
15
45

OUTPUT
Enter 15 numbers: Sorted array: 5 10 15 20 25 30 35 40 45 50 55 60 70 80 90
Enter target: Binary Search Iterations:
low=0, high=14, mid=7, arr[7]=40

QUESTIONS 10 / 10 completed

Binary Search- Iter: ▾

BINARY SEARCH-ITERATIVE

temperatures, indices (0-based), or ""Not found"".

Validate sorted; n<=50.

Sample Input: 6
22 28 31 24 29 32

Sample Output (after sort):
22 24 28 29 31 32

First >=30: index 4 (31)

Last <=25: index 1 (24)

PROGRAM EDITOR

C ▾

Run Save

```
58 for (int i = 0; i < n; i++) {
59     printf("%d%s", arr[i], (i == n - 1) ? "" : " ");
60 }
61 printf("\n");
62
63 int firstIdx = findFirstGreaterEqual(arr, n, 30);
64 if (firstIdx != -1) {
65     printf("First >=30: index %d (%d)\n", firstIdx, arr[firstIdx]);
66 } else {
67     printf("First >=30: Not found\n");
68 }
69
70 int lastIdx = findLastLessEqual(arr, n, 25);
71 if (lastIdx != -1) {
72     printf("Last <=25: index %d (%d)\n", lastIdx, arr[lastIdx]);
73 } else {
74     printf("Last <=25: Not found\n");
75 }
76
77 return 0;
78 }
```

OUTPUT

Sample Output (after sort):
22 24 28 29 31 32
First >=30: index 4 (31)

INPUT

6
22 28 31 24 29 32

SIMATS
ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

Nithiyasri S
192572082RB ▾

QUESTIONS 10 / 10 completed

Linear Search - Ans ▾

LINEAR SEARCH - ANAGRAMS

"Read n strings, use linear search to find all anagrams of a target string (same chars, any order). Display positions (1-based) and matching strings.

Case-insensitive; n<=50.

Sample Input: 5
1) listen silent
2) race care
3) hello world
4) abc cba

PROGRAM EDITOR

C ▾

Run Save

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4
5 void sortString(char *str) {
6     int n = strlen(str);
7     for (int i = 0; i < n - 1; i++) {
8         for (int j = i + 1; j < n; j++) {
9             if (str[i] > str[j]) {
10                 char temp = str[i];
11                 str[i] = str[j];
12                 str[j] = temp;
13             }
14         }
15     }
16 }
17
18 int checkAnagram(char *s1, char *target) {
19     int l1 = strlen(s1);
20     int l2 = strlen(target);
21     if (l1 != l2) return 0;
```

OUTPUT

INPUT

5
listen silent
race care
hello world
abc cba
Xyz
silent

STAMATS ENGINEERING

DS PROGRAMMING

Nithiyasri S
192572082RB

QUESTIONS 10 / 10 completed

Linear Search on A

LINEAR SEARCH ON ARRAY

"Develop a C program to perform Linear Search on an array of N integers. Accept array size dynamically"

Sample Input/Output
Enter size: 5
Array: 10 20 30 40 50
Enter target: 40
Comparison Trace:
10 != 40
20 != 40
30 != 40

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int n, target, i, found = 0, comparisons = 0;
6
7     // Step 1: Accept array size dynamically
8     printf("Enter size: ");
9     if (scanf("%d", &n) != 1 || n <= 0) {
10         printf("Invalid size!\n");
11         return 1;
12     }
13
14     // Dynamically allocate memory for N integers
15     int *arr = (int *)malloc(n * sizeof(int));
16     if (arr == NULL) {
17         printf("Memory allocation failed!\n");
18         return 1;
19     }
20
21     // Step 2: Accept array elements
```

OUTPUT

Enter size: Array: Enter target: Comparison Trace:
10 != 40
20 != 40

INPUT

5
10 20 30 40 50
40

STAMATS ENGINEERING

DS PROGRAMMING

Nithiyasri S
192572082RB

QUESTIONS 10 / 10 completed

Binary Search - Arr

BINARY SEARCH - ARRAY

Write a C program to perform Binary Search on a sorted array of 10 integers. The program should accept the array elements and a target value from the user. It must display the position of the element and the total number of comparisons made.

Sample Input:
Enter 10 sorted

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int arr[10];
5     int target, i;
6     int low = 0, high = 9;
7     int mid;
8     int comparisons = 0;
9     int found = 0;
10    int position = -1;
11
12    // Input the 10 sorted integers
13
14    for (i = 0; i < 10; i++) {
15        scanf("%d", &arr[i]);
16    }
17
18    // Input the target element
19    printf("Enter element to search: ");
20    scanf("%d", &target);
21
```

OUTPUT

Enter element to search: Element 45 found at position 6
Comparisons made: 3

INPUT

5 12 18 23 34 45 56
67 78 90
45

**SIMATS
ENGINEERING**

DS PROGRAMMING

192572082RB

QUESTIONS 10 / 10 completed

Linear Search - Tem

LINEAR SEARCH - TEMPERATURE

"Read n daily temperatures into an array. Use Linear Search to find the last day whose temperature is less than 20°C. Display temperatures, index (0-based), and value. If not found, print "Not found". Validate n ≤ 50.

Sample Input: 7
18 25 21 19 30 17 24

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int n, temp[50];
5     int lastIndex = -1;
6
7     scanf("%d", &n);
8
9     if (n > 50) {
10        printf("Invalid input: n exceeds 50\n");
11        return 0;
12    }
13
14    for (int i = 0; i < n; i++) {
15        scanf("%d", &temp[i]);
16    }
17
18    // Display temperatures
19    for (int i = 0; i < n; i++) {
20        printf("%d ", temp[i]);
21    }
```

OUTPUT

INPUT

7
18 25 21 19 30 17 24

**SIMATS
ENGINEERING**

SIMATS Online Programming Lab

DS PROGRAMMING

Nithiyasri S
192572082RB

QUESTIONS 10 / 10 completed

Linear Search - Prio

LINEAR SEARCH - PRICES

"Read n product prices into an array. Sort ascending using any method. Use linear search to find the first price ≥ ₹500 and the last price ≤ ₹200. Display prices and indices (0-based), or "Not found". Validate sorted order; n ≤ 50.

Sample Input: 7
150 650 220 90 500 780 180

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int main() {
4     int n;
5     scanf("%d", &n);
6
7     int prices[50];
8     for (int i = 0; i < n; i++) {
9         scanf("%d", &prices[i]);
10    }
11
12    // Sort ascending using bubble sort
13    for (int i = 0; i < n - 1; i++) {
14        for (int j = 0; j < n - i - 1; j++) {
15            if (prices[j] > prices[j + 1]) {
16                int temp = prices[j];
17                prices[j] = prices[j + 1];
18                prices[j + 1] = temp;
19            }
20        }
21    }
```

OUTPUT

INPUT

7
150 650 220 90 500 780 180

Sample Output (after sort): 90 150 180 220 500 650 780
First ≥500: index 4 (500)

QUESTIONS 10 / 10 completed

Bin. Search-Iterativ

BIN. SEARCH-
ITERATIVE-TEMP

"Read n daily
temperatures into an
array, sort ascending
using any method.
Use binary search
(iterative) to find first
day $\geq 30^\circ\text{C}$ and last
day $\leq 25^\circ\text{C}$. Display
temperatures,
indices (0-based), or
""Not found".
Validate sorted;
 $n \leq 50$.
Sample Input 6

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Function to sort the array (using Bubble Sort for simplicity, fits n <= 50)
5 void sortArray(int arr[], int n) {
6     for (int i = 0; i < n - 1; i++) {
7         for (int j = 0; j < n - i - 1; j++) {
8             if (arr[j] > arr[j + 1]) {
9                 int temp = arr[j];
10                arr[j] = arr[j + 1];
11                arr[j + 1] = temp;
12            }
13        }
14    }
15 }
16
17 // Iterative Binary Search to find the FIRST index where element >= target
18 int findFirstGreaterOrEqual(int arr[], int n, int target) {
19     int low = 0, high = n - 1;
20     int result = -1;
21     while (low <= high) {
```

OUTPUT

```
22 24 28 29 31 32
First >=30: index 4 (31)
```

INPUT

```
6
22 28 31 24 29 32
```